

Цифровой куратор

Инструкция по развёртыванию на рабочем сервере

для системного администратора организации

Версия платформы	3.3.0
Дата документа	08.10.2026
Целевая среда	MySQL 8.1.0 · Python 3.14 · Apache 2.4 (обратный прокси) · без Docker и без дополнительных сервисов
Версии ПО на сервере	Не обновляются: Python 3.14 и MySQL 8.1.0 остаются как есть
Что получает администратор	Архив kait20-3.3.0.zip (+ файл .sha256) и этот документ

Содержание

1. Как устроена платформа и что она требует
2. Что нужно на сервере
3. Что и как переносить
4. Установка: шаг за шагом
5. Эксплуатация: запуск, логи, резервные копии, обновление
6. Безопасность и требования к окружению
7. Первичная загрузка групп, студентов и кураторов
8. Типичные проблемы и их решение
9. Результаты проверки готовности
10. Приложения: конфигурации (Apache, systemd, резервное копирование)

1. Как устроена платформа

Платформа — **одно приложение**: сервер Python (FastAPI) сам отдаёт и интерфейс (готовые файлы уже внутри архива), и данные (API). Никаких отдельных веб-серверов для статики, очередей, кэшей и планировщиков нет. Apache нужен только как точка входа по HTTPS и обратный прокси.

Звено	Что делает	Адрес
Браузер пользователя	Работает только с вашим доменом; сторонних сайтов, шрифтов и CDN платформа не использует	https://<домен>/
Apache 2.4	HTTPS, перенаправление с http, обратный прокси на приложение	порты 80 и 443
Приложение kait20	Python 3.14, один процесс (служба systemd), интерфейс + API	127.0.0.1:8000 (только локально)
MySQL 8.1.0	Все данные; таблицы создаются и обновляются автоматически при запуске (миграции)	127.0.0.1:3306

- **Запускать нужно ровно один экземпляр приложения.** Счётчик неудачных попыток входа хранится в памяти процесса; несколько экземпляров считали бы его отдельно. Для нагрузки колледжа (около тысячи студентов и сотни кураторов) одного процесса достаточно.
- Планировщика заданий нет: напоминания и периодические задачи выполняются, когда кто-то открывает платформу. Отдельный cron для приложения не нужен (cron нужен только для резервных копий).
- Исходящий доступ в интернет приложению **не нужен** (Sentry по умолчанию выключен).
- Apache должен передавать в приложение **все** адреса без исключения (в том числе страницы интерфейса). Приложение само отличает открытие страницы в браузере от запроса данных по заголовкам браузера (Sec-Fetch-Dest / Accept), поэтому раздавать часть файлов из Apache или подменять ошибки прокси (ProxyErrorOverride) нельзя — страницы перестанут открываться по прямой ссылке или после обновления (F5).

2. Что нужно на сервере

Компонент	Требование	Комментарий
Python	3.14.x (уже установлен, не обновлять)	Приложение не запустится на другой версии — это проверка на старте. Нужен модуль venv (в Debian/Ubuntu — пакет python3.14-venv, если его нет).
MySQL	8.1.0 (уже установлен, не обновлять), кодировка utf8mb4	Используются только стандартные возможности MySQL 8.0+. max_connections — не меньше 60 (пул приложения до 30 соединений + запас для резервного копирования и обслуживания).
Apache	2.4 с модулями proxy, proxy_http, headers, ssl	Если модули не включены: a2enmod proxy proxy_http headers ssl (Debian/Ubuntu) либо LoadModule в httpd.conf (RHEL-семейство).
Сертификат TLS	для домена платформы	Без HTTPS вход работать не будет (см. раздел 6).
Шрифт DejaVu Sans	пакет fonts-dejavu-core (или аналог)	Нужен только для кириллицы в PDF-выгрузках.
Клиент MySQL	mysql и mysqldump	Для создания БД и резервных копий; обычно ставятся вместе с сервером.
Синхронизация времени	chrony или systemd-timesyncd	Платформа считает «сегодня», сроки сдачи и подачи питания по времени сервера.
Ресурсы	1–2 vCPU, 2 ГБ ОЗУ, 5 ГБ диска	Файлы приложения занимают около 4 МБ, виртуальное окружение — несколько сотен МБ; остальное — база и копии.

Устанавливать дополнительно не нужно: Node.js и npm (интерфейс уже собран), Docker, компилятор и заголовочные файлы (все Python-пакеты ставятся готовыми колёсами — проверено для Linux x86_64 и Python 3.14), mod_wsgi, Redis, RabbitMQ, Nginx. Подключения к внешним сервисам не используются.

3. Что и как переносить

3.1. Передаваемый комплект

Что	Откуда	Куда
Архив kait20-3.3.0.zip и kait20-3.3.0.zip.sha256	готовит разработчик: python tools/build_release.py	на сервер, в /tmp
Этот документ	—	администратору
Файлы vhost, службы и резервного копирования (приложения А–С)	docs/deploy в репозитории или текст из приложений	см. раздел 4
Файлы с данными для загрузки (реестр контингента, список кураторов)	передаются отдельно, НЕ через git и НЕ в составе архива	/var/lib/kait20/import

Внутри архива две папки: **backend** (приложение: код, готовый интерфейс в backend/static, миграции БД, скрипты загрузки данных, список зависимостей requirements.txt, шаблон настроек .env.example) и **frontend** (исходники интерфейса — на сервере не нужны, их можно не распаковывать). Тестов, файлов с реальными данными и секретов в архиве нет.

3.2. Что НЕ переносится

- Файл .env разработки и любые пароли тестового окружения. Для рабочего сервера создаются **новые** секреты.
- Каталог .venv, node_modules, кэши — виртуальное окружение создаётся на самом сервере.
- Файлы с реальными ФИО в репозиторий не попадают (они в .gitignore) и передаются администратору отдельно защищённым каналом.

3.3. Данные: два сценария

Сценарий	Когда выбирать	Что делать с ключами
А. Чистая установка (рекомендуется)	Рабочий сервер стартует с пустой базы; группы, студентов и кураторов загружают из файлов (раздел 7).	Создать новые JWT_SECRET и DOSSIER_ENCRYPTION_KEY, сохранить ключ шифрования в сейфе отдельно от резервных копий БД.
Б. Перенос действующей тестовой базы	На тестовой платформе уже заведены пользователи, отметки посещаемости, досье, которые нужно сохранить.	DOSSIER_ENCRYPTION_KEY должен быть тем же , что на тестовой платформе, иначе особые поля досье не расшифровать. JWT_SECRET — новый (все сессии пользователей сбросятся, это нормально).

Порядок для сценария Б: на источнике снять дамп mysqldump --single-transaction --no-tablespaces --routines --triggers --default-character-set=utf8mb4 <база> | gzip > kait20.sql.gz, передать файл защищённым каналом, на сервере создать пустую БД (п. 4.3) и выполнить zcat kait20.sql.gz | mysql kait20 **до первого запуска приложения**; затем запустить приложение — оно доведёт схему до актуальной версии автоматически. Сначала отработайте перенос на тестовой копии: версия MySQL на источнике может отличаться от 8.1.0.

Важно. Ключ DOSSIER_ENCRYPTION_KEY нельзя терять и менять: без него сохранённые особые данные досье не прочитать, а резервная копия БД без ключа для этих полей бесполезна. Храните ключ отдельно от копий базы.

4. Установка: шаг за шагом

Команды приведены для Linux (Debian/Ubuntu); для RHEL-семейства отличаются только команды установки пакетов и пути конфигурации Apache. Здесь и далее `kurator.example.ru` — замените на реальный домен платформы.

4.1. Проверка версий (ничего не обновляя)

```
python3.14 --version          # должно быть Python 3.14.x
mysql --version               # должно быть 8.1.0
apache2ctl -v                 # 2.4.x (RHEL: httpd -v)
apache2ctl -M | grep -E 'proxy|headers|ssl' # нужны: proxy, proxy_http, headers, ssl
```

4.2. Пользователь и каталоги

```
sudo useradd --system --home /opt/kait20 --shell /usr/sbin/nologin kait20
sudo mkdir -p /opt/kait20 /var/lib/kait20/import /var/backups/kait20
sudo chown kait20:kait20 /opt/kait20 /var/lib/kait20 /var/lib/kait20/import /var/backups/kait20
sudo chmod 700 /var/lib/kait20/import /var/backups/kait20
```

4.3. База данных

```
-- Выполнить от имени администратора MySQL (mysql -u root -p)
CREATE DATABASE kait20 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'kait20'@'localhost' IDENTIFIED BY '<пароль>';
CREATE USER 'kait20'@'127.0.0.1' IDENTIFIED BY '<пароль>';
GRANT ALL PRIVILEGES ON kait20.* TO 'kait20'@'localhost';
GRANT ALL PRIVILEGES ON kait20.* TO 'kait20'@'127.0.0.1';
FLUSH PRIVILEGES;
```

Пароль в команды вставляется один раз и дальше хранится только в `.env` (п. 4.6) и в `~/my.cnf` пользователя `kait20` (для резервных копий). Миграции при запуске создают и изменяют таблицы, поэтому пользователю приложения нужны права на всю БД `kait20` (как выше).

4.4. Распаковка и проверка целостности

```
cd /tmp
sha256sum -c kait20-3.3.0.zip.sha256          # должно быть: OK
sudo -u kait20 unzip -q kait20-3.3.0.zip 'backend/*' -d /opt/kait20
ls /opt/kait20/backend                         # app, alembic, scripts, static, requirements.txt, .env.example
```

4.5. Виртуальное окружение и зависимости

```
cd /opt/kait20/backend
sudo -u kait20 python3.14 -m venv .venv
sudo -u kait20 .venv/bin/pip install --require-hashes -r requirements.txt
```

Ключ `--require-hashes` заставляет `pip` проверять подлинность каждого пакета по хэшу из `requirements.txt`. Сервер при установке обращается к индексу пакетов (`pypi.org`) — если прямого доступа в интернет нет, см. последнюю строку таблицы в разделе 8.

4.6. Настройки (.env)

```
sudo -u kait20 cp /opt/kait20/backend/.env.example /opt/kait20/backend/.env
sudo chmod 600 /opt/kait20/backend/.env
# Генерация секретов (вывод вставить в .env; ключ шифрования — ещё и в сейф):
PY=/opt/kait20/backend/.venv/bin/python
$PY -c "import secrets; print(secrets.token_urlsafe(48))"
$PY -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Содержимое `.env` для рабочего сервера:

```
# /opt/kait20/backend/.env    (владелец kait20, права 600)
ENVIRONMENT=production

DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=kait20
DB_PASSWORD=<пароль пользователя БД>
DB_NAME=kait20

# Случайная строка не короче 32 символов (команда генерации – в п. 4.6)
JWT_SECRET=<сгенерировать>
# Ключ шифрования особых полей досье (здоровье, соц. статус, учёт). Хранить ОТДЕЛЬНО от БД и не менять!
DOSSIER_ENCRYPTION_KEY=<сгенерировать или взять действующий – см. п. 3.3>

# Логин и ВРЕМЕННЫЙ пароль первого администратора; при первом входе платформа потребует сменить пароль
ADMIN_USERNAME=admin
ADMIN_PASSWORD=<временный пароль>

# Перед приложением стоит ровно один прокси (Apache)
TRUSTED_PROXY_COUNT=1
NOTIFICATION_TIMEZONE=Europe/Moscow
# Пул соединений с БД (по умолчанию 10 + 20 запасных); должен укладываться в max_connections MySQL
# DB_POOL_SIZE=10
# DB_MAX_OVERFLOW=20
```

4.7. Пробный запуск вручную

```
cd /opt/kait20/backend
sudo -u kait20 env HOST=127.0.0.1 .venv/bin/python -m scripts.entrypoint
# Ожидаемо: «База доступна» → «Миграции...» → «Справочники...» → «Запуск приложения...»
# В другом окне: curl http://127.0.0.1:8000/health → {"status":"ok"}
# Остановить: Ctrl+C
```

Скрипт запуска сам дожидается базы, применяет миграции, создаёт справочники и администратора и запускает приложение. Если JWT_SECRET слабый или версия Python не 3.14, он остановится с понятным сообщением.

4.8. Служба systemd

Файл — в приложении В (docs/deploy/kait20.service). Установка:

```
sudo cp kait20.service /etc/systemd/system/kait20.service
sudo systemctl daemon-reload
sudo systemctl enable --now kait20
sudo systemctl status kait20          # active (running)
journalctl -u kait20 -n 50 --no-pager # журнал запуска
```

4.9. Apache

Файл — в приложении А (docs/deploy/kait20-apache.conf): впишите домен и пути к сертификату.

```
sudo cp kait20-apache.conf /etc/apache2/sites-available/kait20.conf
# (RHEL-семейство: /etc/httpd/conf.d/kait20.conf)
sudo a2enmod proxy proxy_http headers ssl
sudo a2ensite kait20
sudo apache2ctl configtest          # должно быть: Syntax OK
sudo systemctl reload apache2
```

4.10. Проверка после установки (чек-лист приёмки)

№	Проверка	Ожидаемый результат
1	systemctl status kait20; journalctl -u kait20	active (running), в журнале нет ошибок
2	curl http://127.0.0.1:8000/health на сервере	{"status":"ok"}
3	Порт 8000 снаружи (с другого компьютера)	НЕ отвечает (слушает только 127.0.0.1)
4	http://<домен>/ в браузере	Перенаправление на https, открывается экран входа
5	Вход admin с временным паролем из .env	Платформа просит задать новый пароль; после смены открывается рабочий экран
6	Обновить страницу (F5) на внутренней странице, например /admin	Открывается интерфейс (не текст с JSON и не ошибка)
7	Админка → Витрины, любая выгрузка в Excel и PDF	Файл скачивается, кириллица в PDF читается
8	Админка → Пользователи → профиль → «Выдать пароль»	Временный пароль показывается
9	Остановить приложение (systemctl stop kait20) и открыть сайт	Apache отвечает 503/502; после запуска всё работает без других действий
10	Перезагрузить сервер	kait20, MySQL и Apache поднимаются сами; сайт открывается

4.11. Первый вход

Логин — значение ADMIN_USERNAME, пароль — ADMIN_PASSWORD из .env. Пароль **временный**: при первом входе платформа требует задать новый. После смены пароля строку ADMIN_PASSWORD из .env рекомендуется удалить (файл прочитан только при создании администратора).

5. Эксплуатация

5.1. Ежедневное

Задача	Команда
Состояние	systemctl status kait20
Журнал приложения (последние 100 строк / в реальном времени)	journalctl -u kait20 -n 100 --no-pager / journalctl -u kait20 -f
Перезапуск (после изменения .env)	sudo systemctl restart kait20
Журналы Apache	/var/log/apache2/kait20-error.log и kait20-access.log (RHEL: /var/log/httpd)
Проверка доступности (для мониторинга)	GET https://<домен>/health → 200

5.2. Резервное копирование

Скрипт — приложение C (docs/deploy/kait20-backup.sh). Делает сжатый дамп БД, проверяет, что архив читается и содержит таблицы, хранит 30 дней. Настройка:

```
sudo cp kait20-backup.sh /opt/kait20/kait20-backup.sh && sudo chmod 755 /opt/kait20/kait20-backup.sh
# учётные данные для mysqldump (чтобы пароль не светился в списке процессов):
sudo -u kait20 sh -c 'printf "[client]\nuser=kait20\npassword=<пароль БД>\n" > ~/.my.cnf'
sudo -u kait20 chmod 600 ~/.my.cnf
# ежедневно в 02:30:
echo '30 2 * * * kait20 /opt/kait20/kait20-backup.sh >>/var/log/kait20-backup.log 2>&1' \
| sudo tee /etc/cron.d/kait20-backup
```

Копии БД нужно **выносить с этого сервера** (сетевая папка, ленточное хранилище, другой сервер): потеря диска уничтожит и базу, и копии. Ключ DOSSIER_ENCRYPTION_KEY и файл .env копируются отдельно и хранятся в сейфе.

5.3. Восстановление (проверить хотя бы один раз до ввода в эксплуатацию)

```
sudo systemctl stop kait20
mysql -e "DROP DATABASE kait20; CREATE DATABASE kait20 \
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
zcat /var/backups/kait20/kait20-<дата>.sql.gz | mysql kait20
sudo systemctl start kait20
```

5.4. Обновление версии платформы

```
# 1. резервная копия БД и копия текущей папки приложения
sudo -u kait20 /opt/kait20/kait20-backup.sh
sudo cp -a /opt/kait20/backend /opt/kait20/backend.prev
# 2. остановить, заменить код (сохранив .env и .venv), обновить зависимости, запустить
sudo systemctl stop kait20
cd /tmp && sha256sum -c kait20-<версия>.zip.sha256
sudo -u kait20 unzip -q kait20-<версия>.zip 'backend/*' -d /tmp/kait20-new
sudo rsync -a --exclude .env --exclude .venv /tmp/kait20-new/backend/ /opt/kait20/backend/
sudo chown -R kait20:kait20 /opt/kait20/backend
cd /opt/kait20/backend && sudo -u kait20 .venv/bin/pip install --require-hashes -r requirements.txt
sudo systemctl start kait20 && curl -s http://127.0.0.1:8000/health
```

Миграции БД применяются при запуске автоматически. **Откат:** миграции работают только вперёд, поэтому откат — это возврат прежней папки (backend.prev) вместе с восстановлением БД из копии, снятой перед обновлением (п. 5.3).

6. Безопасность и требования к окружению

Что уже сделано в приложении (проверено): сессия — HttpOnly-cookie с флагами Secure и SameSite=Strict; защита от подбора пароля — блокировка по связке «адрес + логин» (учётную запись мошенник заблокировать не может), для администраторов и руководителей лимит строже; заголовки HSTS, Content-Security-Policy (только свои ресурсы), X-Frame-Options; Swagger и схема API в рабочем режиме выключены; пароли хранятся в виде хэшей Argon2; особые данные досье шифруются; просмотр особых данных пишется в журнал.

От администратора требуется:

- **Только HTTPS.** Cookie сессии помечена Secure: по обычному http браузер её отбросит, и вход «не запомнится». Если на время запуска сертификата нет, в .env можно временно добавить SESSION_COOKIE_SECURE=false — с реальными данными так работать нельзя.
- Приложение слушает только 127.0.0.1 (так настроена служба); порт 8000 в файрволе наружу не открывать. MySQL — тоже только локально (bind-address=127.0.0.1), если сервер БД не вынесен отдельно.
- Права: .env — 600 (владелец kait20); ~/.my.cnf — 600; /var/lib/kait20/import и /var/backups/kait20 — 700.
- TRUSTED_PROXY_COUNT=1, если перед приложением ровно один Apache. Если между Apache и пользователями есть ещё балансировщик — значение 2; если приложение вдруг доступно напрямую без прокси — 0. Неверное значение ломает лимит попыток входа (все пользователи сольются в один адрес).
- Файлы с реальными ФИО (реестр, списки кураторов) после загрузки удалять с сервера (shred -и или стандартное безопасное удаление).
- Регулярные обновления ОС и Apache — как обычно; **Python 3.14 и MySQL 8.1.0 не обновлять** без согласования с разработчиком.

7. Первичная загрузка групп, студентов и кураторов

В платформе нет экрана массовой загрузки контингента; загрузка выполняется **командами на сервере** из файлов Excel и текста. Все команды по умолчанию работают в режиме проверки (ничего не пишут), запись включается ключом - -apply; повторный запуск тех же данных ничего не меняет. Загрузка не требует обновления версии платформы и остановки сайта.

7.1. Форматы файлов

Файл	Формат	Что обязательно
registry.xlsx — реестр контингента	Excel, первый лист; строка заголовка ищется по названиям колонок, остальные колонки (телефон, почта) игнорируются и в базу не попадают	Колонки: ФИО ; Статус обучения («Обучается» или «В академическом отпуске»); Учебная группа (например ИИ112-26 — хвост «-26» или «.8» отбрасывается); Адрес площадки (по нему определяется отделение; известные адреса зашиты в скрипт); Курс обучения («1 курс»)
curators.tsv — кураторы	Текст UTF-8, значения через табуляцию, одна строка = одна группа; один куратор может вести несколько групп	ФИО → код группы → отделение. Куратор создаётся без пароля, логин — фамилия.инициал латиницей

7.2. Порядок

```
# файлы положить в /var/lib/kait20/import (владелец kait20, права 600)
sudo -u kait20 /opt/kait20/kait20-backup.sh          # 0. резервная копия — ОБЯЗАТЕЛЬНО
cd /opt/kait20/backend
RUN="sudo -u kait20 .venv/bin/python -m"
DATA=/var/lib/kait20/import
$RUN scripts.import_registry $DATA/registry.xlsx    # 1. реестр: проверка (ничего не пишет)
$RUN scripts.import_registry $DATA/registry.xlsx --apply # 2. реестр: запись
$RUN scripts.import_curators $DATA/curators.tsv      # 3. кураторы: проверка
$RUN scripts.import_curators $DATA/curators.tsv --apply # 4. кураторы: запись
$RUN scripts.hide_groups --apply                     # 5. скрыть группы со строчной «о» в коде
sudo shred -u $DATA/*                               # 6. удалить файлы с ФИО
```

Реестр удаляет. Студенты, которых нет в файле реестра, **полностью удаляются** из базы вместе с отметками посещаемости, периодами отсутствия и записями о группах; их ФИО в журнале аудита затираются необратимо. Поэтому: сначала резервная копия, потом проверка (без --apply) и сверка отчёта с ожидаемыми цифрами, и только потом запись. Кураторов и их назначения реестр не трогает.

После загрузки кураторов у них **нет паролей**: администратор платформы выдаёт временный пароль каждому (Админка → Пользователи → профиль → «Выдать пароль»); при первом входе куратор задаёт свой.

Варианты развития механизма загрузки (экран в платформе, единый шаблон Excel, выдача паролей списком) описаны в отдельном документе «Загрузка контингента и кураторов: анализ и варианты».

8. Типичные проблемы и их решение

Симптом	Причина	Что сделать
Приложение не запускается: «JWT_SECRET не задан или слишком короткий»	Секрет пустой/предсказуемый	Сгенерировать JWT_SECRET (п. 4.6), перезапустить
«Требуется Python >=3.14,<3.15»	venv создан другой версией Python	Пересоздать venv командой <code>python3.14 -m venv</code>
«DOSSIER_ENCRYPTION_KEY некорректен»	Ключ испорчен при копировании	Ключ Fernet — 44 символа, заканчивается на «=»; вставить без пробелов и кавычек
Apache отвечает 502/503	Служба kait20 не запущена или падает	<code>systemctl status kait20; journalctl -u kait20 -n 100</code>
Вход проходит, но сразу выбрасывает на экран входа	Сайт открыт по http, а cookie помечена Secure	Настроить HTTPS (раздел 6)
Обновление страницы (F5) показывает текст «Нужна авторизация» или ошибку	Apache подменяет ответы или не передаёт заголовки браузера	Убрать ProxyErrorOverride и любые Rewrite/Alias, пропускающие часть адресов мимо прокси
«Слишком много попыток входа» у всех сразу	Неверный TRUSTED_PROXY_COUNT: приложение видит всех с одного адреса	Поставить число прокси перед приложением (обычно 1)
В логе «QueuePool limit ... reached»	Исчерпан пул соединений с БД	Увеличить DB_POOL_SIZE/DB_MAX_OVERFLOW и max_connections MySQL, перезапустить
В PDF вместо кириллицы квадраты	Нет шрифта DejaVu Sans	Установить fonts-dejavu-core, перезапустить службу
Время «сегодня» отличается на несколько часов	Часовой пояс/синхронизация	Проверить NOTIFICATION_TIMEZONE и chrony; перезапустить
pip install не может скачать пакеты	Нет доступа к pypi.org	Скачать колёса на компьютере с интернетом: <code>pip download -r requirements.txt --only-binary=:all: --python-version 3.14 -d wheels</code> , перенести папку и ставить с <code>--no-index --find-links wheels --require-hashes</code>

9. Результаты проверки готовности

Перед подготовкой этого документа проведена проверка репозитория, конфигурации и сборки. Итог:

Область	Результат
Сборка архива для сервера (tools/build_release.py)	Найдено и исправлено: служебный комментарий в коде содержал название стороннего хостинга, из-за чего сборка архива останавливалась. После исправления архив собирается (344 файла, 2,4 МБ), без тестов, секретов и Docker-файлов.

Область	Результат
Зависимости на Python 3.14	Все 41 пакет из requirements.txt получены готовыми колёсами для Linux x86_64 / CPython 3.14: компилятор на сервере не нужен. Для другой архитектуры (например ARM) нужна отдельная проверка.
Работа за Apache	Проверено на живом Apache 2.4 с TLS: проксирование, перенаправление http→https, открытие страниц по прямому адресу и обновление (F5), вход (cookie Secure + HttpOnly + SameSite=Strict), заголовки HSTS и CSP, загрузки Excel. Приложены проверенные файлы конфигурации.
Вход по http без сертификата	Раньше при ENVIRONMENT=production вход по http не работал вообще (браузер отбрасывал Secure-cookie). Добавлена настройка SESSION_COOKIE_SECURE (по умолчанию включено; временно можно выключить).
Загрузка файлов больше 5 МБ	Через Apache отказ приходил общей ошибкой 500 вместо понятного сообщения. Теперь размер проверяется в интерфейсе заранее.
Резервное копирование	В скрипте добавлен --no-tablespaces: без него mysqldump на MySQL 8.0+ под обычным пользователем БД завершается ошибкой (нужна привилегия PROCESS).
Скрипты загрузки контингента и кураторов	Проверены на синтетических данных: пробный режим, запись, повторный запуск (идемпотентность). Ограничения описаны в разделе 7.
MySQL 8.1.0	Автоматические тесты платформы в CI идут на MySQL 8.4 и на SQLite; на 8.1.0 они не запускались. В коде и миграциях используются только стандартные возможности MySQL 8.0+ (без JSON-колонок, функциональных индексов и специфичного синтаксиса 8.4), поэтому несовместимости не ожидается, но первый запуск на тестовой копии БД обязателен (п. 4.7).
Не проверялось	Установка на самом рабочем сервере и под systemd (шаблон службы составлен по документации, не запускался на реальной машине); Windows-сервер; ARM; нагрузочные испытания на реальном числе пользователей.

Приложение А. Виртуальный хост Apache (docs/deploy/kait20-apache.conf)

```
# Виртуальный хост Apache для платформы «Цифровой куратор».
# Apache — только HTTPS-вход и обратный прокси; интерфейс и API отдаёт
# одно приложение на 127.0.0.1:8000.
# Нужные модули: proxy, proxy_http, headers, ssl (a2enmod proxy proxy_http headers ssl)
# Замените: kurator.example.ru, пути к сертификату и ключу.

<VirtualHost *:80>
    ServerName kurator.example.ru
    Redirect permanent / https://kurator.example.ru/
</VirtualHost>

<VirtualHost *:443>
    ServerName kurator.example.ru

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/kurator.example.ru.crt
    SSLCertificateKeyFile /etc/ssl/private/kurator.example.ru.key
    SSLProtocol -all +TLSv1.2 +TLSv1.3

    # BCE пути — в приложение: оно само решает, отдать страницу интерфейса или данные API
    # (по заголовкам браузера Sec-Fetch-Dest/Асцепт). Отдельно раздавать файлы из Apache и
    # подменять ошибки (ProxyErrorOverride) нельзя — интерфейс перестанет открываться по прямым адресам.
    ProxyRequests Off
    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8000/ nocanon
    ProxyPassReverse / http://127.0.0.1:8000/

    # Схема исходного запроса: по ней приложение ставит Secure-cookie и HSTS.
    RequestHeader set X-Forwarded-Proto "https"

    # Выгрузки Excel/Word/PDF по большим спискам могут идти десятки секунд.
    ProxyTimeout 180
    # Размер загружаемых файлов (Excel с досье — до 5 МБ) проверяет само приложение и отвечает
    # понятным сообщением. LimitRequestBody в Apache не ставить: вместо отказа он даёт ошибку 500.

    # Заголовки безопасности (CSP, HSTS, X-Frame-Options) выставляет приложение — не дублировать.
    Header always unset X-Powered-By

    ErrorLog ${APACHE_LOG_DIR}/kait20-error.log
    CustomLog ${APACHE_LOG_DIR}/kait20-access.log combined
</VirtualHost>
```

Приложение В. Служба systemd (docs/deploy/kait20.service)

```
# /etc/systemd/system/kait20.service
[Unit]
Description=Цифровой куратор (КАИТ №20)
After=network.target mysql.service mysqld.service mariadb.service
Wants=network.target

[Service]
Type=simple
User=kait20
Group=kait20
WorkingDirectory=/opt/kait20/backend
# Секреты и настройки читает само приложение из /opt/kait20/backend/.env (права 600, владелец kait20).
ExecStart=/opt/kait20/backend/.venv/bin/python -m scripts.entrypoint
Environment=PYTHONUNBUFFERED=1
Environment=HOST=127.0.0.1
Environment=PORT=8000
# Запускать РОВНО ОДИН экземпляр: лимит попыток входа хранится в памяти процесса.
Restart=on-failure
RestartSec=5
# Миграции и справочники при старте занимают до минуты на большой базе.
TimeoutStartSec=300

# Ограничения: приложению нужны только своя папка и чтение .env.
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ProtectHome=true

[Install]
WantedBy=multi-user.target
```

Приложение С. Резервное копирование (docs/deploy/kait20-backup.sh)

```
#!/bin/bash
# Ежедневная резервная копия БД платформы. Запуск из cron от пользователя kait20:
# 30 2 * * * /opt/kait20/kait20-backup.sh
# Учётные данные БД берутся из ~/.my.cnf пользователя kait20 (права 600),
# чтобы пароль не светился в списке процессов:
# [client]
# user=kait20
# password=...
set -euo pipefail
DEST=/var/backups/kait20
KEEP_DAYS=30
mkdir -p "$DEST"
STAMP=$(date +%Y%m%d-%H%M)
mysqldump --single-transaction --no-tablespaces --routines --triggers \
--default-character-set=utf8mb4 kait20 | gzip > "$DEST/kait20-$STAMP.sql.gz"
# Проверка, что архив читается и в нём есть данные (пустой дамп – тоже сбой).
gzip -t "$DEST/kait20-$STAMP.sql.gz"
test "$(zcat "$DEST/kait20-$STAMP.sql.gz" | grep -c 'CREATE TABLE')" -ge 20
find "$DEST" -name 'kait20-*.sql.gz' -mtime +"$KEEP_DAYS" -delete
echo "OK: $DEST/kait20-$STAMP.sql.gz"
```